

IntentCloud

Intent-Aware Cognitive Cloud Memory System

Final Product Requirements Document — Version 4.0

Capstone Project, Phase 1 — 7th Semester, 2026

Sujith Putta (ENG23CT0058) · K Vikas Aneesh Reddy (ENG23CT0052) · Mokshith Karnati (ENG23CT0053) | Guide: Dr. Ramandeep Kaur

1. Document Status — What Changed Since the Original PRD

This document supersedes the original IntentCloud PRD. It consolidates decisions made across the literature survey, the Review-0 presentation, and the refined-objectives note into one authoritative, end-to-end specification of what the team is building, why, and on what schedule.

Version 4.0 responds directly to Review-0 feedback (38/50): it makes hardware-acceleration portability explicit (Apple Silicon MPS / NVIDIA CUDA / CPU fallback), assigns named individual tasks to all three team members for every week of the build plan rather than only the current week, and adds several new features and a more rigorous retrieval algorithm (Reciprocal Rank Fusion) to strengthen the technical depth judges asked for. See §15 for a point-by-point response to the review feedback.

| Area                         | Original PRD                                         | Final (this document)                                                                                                                                                                                                            |
|------------------------------|------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Vector Index                 | FAISS (in-memory, dense-only)                        | Qdrant, embedded on the Pi — dense + sparse hybrid, with metadata persistence                                                                                                                                                    |
| Retrieval Strategy           | Dense semantic search only, top-3 by vector distance | Hybrid (dense + BM25-style sparse) with cross-encoder reranking before top-3                                                                                                                                                     |
| Success Metric               | Response under 15 seconds; no formal accuracy target | ≥85% top-3 accuracy on 30–50 queries over 150+ files, benchmarked against keyword-only and dense-only baselines                                                                                                                  |
| Grounding                    | Problem/solution framing only                        | Every design choice traced to a specific gap in a 19-paper literature survey (see §6)                                                                                                                                            |
| Frontend                     | Plain HTML5 / CSS3 / vanilla JS                      | Next.js (App Router) + Tailwind CSS, statically exported and served from the Pi                                                                                                                                                  |
| Planning Detail              | Phase-level plan only                                | Week-by-week build plan for all of Phase 1, with setup commands and per-week deliverables (§7.7)                                                                                                                                 |
| Implementation Start         | Coding began in Week 3–4, after scaffolding          | Pulled forward to Week 1: env/tool setup and Review-0 happen the same week (§7.7)                                                                                                                                                |
| Hardware Acceleration        | Not specified; CPU-only assumed                      | Runtime auto-detect: Apple Silicon (MPS/Metal) on Mac, CUDA on NVIDIA/Windows laptops, CPU fallback otherwise — identical code path for all three members’ machines                                                              |
| New Features (Post Review-0) | None                                                 | OCR fallback (Tesseract) for scanned files, duplicate/near-duplicate detection at upload, Reciprocal Rank Fusion (RRF) score merging, explainable match-snippet highlighting, confidence-threshold “no confident match” response |
| Weekly Reporting Detail      | Team-level bullets only, current week detailed       | Every week (1–9) in §7.7 lists named, individual tasks for all three members, not just shared team bullets                                                                                                                       |
| Review-0 Outcome             | N/A (not yet delivered)                              | 38/50 — gaps addressed point-by-point in §15, Response to Review-0 Feedback                                                                                                                                                      |

2. Executive Summary

IntentCloud is a self-hosted, privacy-preserving file storage and retrieval system that lets a user find a document by describing what it's about, in plain language, instead of remembering its filename or folder. A locally run language model (Phi-3 Mini) interprets the query,

sentence embeddings represent every stored document's meaning, and a hybrid dense+sparse search — reranked by a cross-encoder — returns the correct file, not a summary of it, within roughly 15 seconds, entirely on consumer-grade hardware with no third-party API calls.

The system splits work across two nodes: a laptop acts as the “cognitive layer” (intent parsing, embeddings, hybrid retrieval, reranking), and a Raspberry Pi 4B acts as the always-on “memory layer” (Qdrant hybrid index, USB file storage, and a Cloudflare Tunnel that exposes one public HTTPS URL). This satisfies the capstone's 50% hardware-implementation requirement while keeping the whole pipeline private and cost-free to run.

### 3. Problem Statement

---

- Students, researchers, and professionals routinely cannot locate documents because they don't remember exact filenames, folder locations, or the keywords used at upload time.
- Keyword search degrades as file volume grows — a single term like “Kafka” can return dozens of irrelevant matches while missing semantically related files that never mention the word.
- Existing AI-powered retrieval tools (ChatGPT-for-PDF, cloud RAG products) depend on commercial LLM APIs, which raises data-privacy concerns, imposes recurring costs, and requires an internet connection.
- No existing solution combines intent-aware retrieval, fully local AI processing, secure edge storage, and remote accessibility in one self-hosted, consumer-hardware-friendly package (literature survey, §6).

IntentCloud addresses this gap directly: a privacy-preserving, self-hosted, intent-aware semantic file-retrieval system that works entirely offline except for the outbound tunnel needed for remote access.

### 4. Refined Objectives

---

1. Achieve  $\geq 85\%$  top-3 retrieval accuracy on natural-language queries, tested on a 150+ file mixed PDF/DOCX/TXT corpus, using a hybrid pipeline of local intent parsing, semantic embeddings, and keyword matching.
2. Run the complete pipeline — text extraction, embedding, intent parsing, retrieval, and reranking — fully on local hardware, with zero calls to third-party LLM APIs.
3. Deliver the original requested file (not a generated summary) in roughly 15 seconds end-to-end on CPU-only hardware.
4. Deploy upload, indexing, hybrid retrieval, and serving as a single self-hosted service reachable through one public HTTPS URL via Cloudflare Tunnel.
5. Empirically benchmark the hybrid, reranked pipeline against a keyword-only baseline and a dense-only baseline, so each component's contribution (hybrid retrieval, reranking) is measured, not assumed.
6. Ensure inference runs efficiently across the team's heterogeneous laptop hardware — Apple Silicon (via MPS/Metal), NVIDIA GPUs on Windows (via CUDA), and CPU-only machines as a fallback — through a single runtime backend-detection layer, with no code changes to the retrieval pipeline across machines, directly addressing the fixed-hardware-configuration gap reported in [18], [19].

### 5. System Architecture

---

The architecture separates intelligence from storage across two physical nodes connected over the local network, with the Pi exposed to the internet only via an outbound Cloudflare Tunnel.

#### 5.1 Laptop — Cognitive Layer

- Ollama + Phi-3 Mini: parses the user's natural-language query into structured intent (topic/keyword JSON).
- sentence-transformers (all-MiniLM-L6-v2): generates 384-dimension embeddings for uploaded documents and queries.
- Hybrid retrieval engine: runs dense similarity search and BM25-style sparse/keyword search in parallel, then reranks the merged candidates with a lightweight cross-encoder.
- Hardware-acceleration backend: Ollama and sentence-transformers auto-detect and use Apple Silicon MPS/Metal on macOS laptops and NVIDIA CUDA on Windows/Linux laptops with a compatible GPU, falling back to CPU-only inference otherwise — the same codebase runs unmodified on all three team members' machines (two Mac, one Windows/CUDA), which is verified in Week 1 setup (§7.7).

#### 5.2 Raspberry Pi 4B — Memory Layer

- Qdrant (embedded mode): stores dense + sparse vectors and metadata together, and executes Qdrant's native hybrid-query fusion — replacing the originally proposed FAISS index, which supports dense search only and has no built-in keyword layer or persistence.
- FastAPI: exposes upload/search/download REST endpoints and serves the statically-exported Next.js UI; files persist on an attached USB drive.

- Cloudflare Tunnel: exposes one public HTTPS URL with no port-forwarding and no static IP, so the system is reachable “from anywhere.”

### 5.3 End-to-End Request Flow

1. Upload doc (PDF / DOCX / TXT) via the Next.js web app UI.
2. Extract text (PyMuPDF for PDF, python-docx for DOCX).
3. Generate sentence-vector embeddings.
4. Analyze intent via Phi-3 Mini for search-time queries.
5. Run hybrid search (dense + sparse) inside Qdrant.
6. Rerank the merged candidates with a cross-encoder; return the top-3.
7. User views and downloads the file directly from Pi/USB storage.

(Architecture and workflow diagrams — Figures 1 and 2 — accompany the Review-1 slide deck and are referenced here rather than reproduced, to keep this document text-first and under five pages.)

### 5.4 New Features & Algorithmic Enhancements (Added After Review-0 Feedback)

Review-0 feedback (38/50) flagged the pipeline as methodologically thin beyond “embed, then vector-search.” The following features and algorithmic steps were added to give the system more technical depth, and are folded into Phases 1, 2, and 4 of the implementation plan (§7):

- **OCR fallback (Tesseract)** at upload time, so text extracted from scanned or image-only PDF pages (which PyMuPDF cannot read directly) is still indexed, closing a gap the original pipeline silently failed on.
- **Duplicate / near-duplicate detection** before indexing: a new upload is embedded, compared by cosine similarity against existing Qdrant vectors, and flagged (not silently stored) if similarity exceeds 0.95 — avoiding redundant storage and duplicate hits polluting search results.
- **Reciprocal Rank Fusion (RRF) for hybrid score merging**: dense and sparse candidate rankings are merged using the RRF formula ( $\text{score}(d) = \sum 1/(k + \text{rank}_i(d))$ ,  $k=60$ ) before the cross-encoder reranks the fused list. This replaces the earlier, under-specified “Qdrant fuses the scores” description with a concrete, citable fusion algorithm.
- **Explainable relevance — matched-snippet highlighting**: each of the top-3 results in the UI shows the specific sentence or passage that most closely matched the query (highest-similarity sentence within the document), giving the human-interpretable “why this matched” explanation that judges asked for and that [3] was found to be missing.
- **Confidence threshold with graceful no-match response**: if the top reranked result scores below a tuned confidence threshold, the system returns “No confident match found” instead of forcing a low-relevance top-3, avoiding the false-positive failure mode common in the dense-only RAG baselines reviewed in §6.

## 6. Literature-Driven Design Rationale

The 19-paper survey covering semantic retrieval, RAG, intent classification, and edge AI surfaced eight recurring gaps; each shapes a concrete IntentCloud design decision:

- Cloud-API dependence and privacy exposure in commercial RAG systems → the entire pipeline runs on local hardware, with no document content leaving the network.
- Systems that return a synthesized answer instead of the source file → IntentCloud always returns and serves the original file.
- Intent classification restricted to narrow, closed-set categories → Phi-3 Mini performs open-ended, free-text intent parsing rather than fixed-category classification.
- High-accuracy systems (e.g., dual-A100, 50 GB memory-scale indexes) requiring infrastructure beyond individual reach → the target corpus (150–200 files) is sized so Qdrant's embedded mode runs comfortably on a single Pi 4B.
- Foundational libraries (FAISS, transformer embeddings) that provide algorithms but no deployable, self-hosted system → IntentCloud ships upload, indexing, retrieval, and serving as one integrated service.
- Dense-only retrieval missing exact filename/keyword hits → hybrid dense+sparse search plus reranking, directly targeting this failure mode.
- Fixed hardware configuration assumed by most reviewed systems [18], [19] → a runtime backend-detection layer selects Apple Silicon MPS/Metal, NVIDIA CUDA, or CPU automatically, so the identical pipeline runs across the team's mixed Mac/Windows laptops without portability code.

## 7. Implementation Plan

### Phase 1 — Data Ingestion & Preprocessing

Browser upload of PDF/DOCX/TXT; fully offline text extraction via PyMuPDF and python-docx, avoiding any per-document API cost. If PyMuPDF returns near-empty text (a scanned/image-only page), a Tesseract OCR fallback runs on that page automatically, so scanned documents are still indexed instead of silently failing extraction.

## Phase 2 — Semantic Representation & Indexing

Embeddings via sentence-transformers (all-MiniLM-L6-v2); indexed in Qdrant (embedded, on-Pi) rather than FAISS, so vectors, metadata, and hybrid scoring live in one system without a separate metadata store. Before a new vector is upserted, it is compared by cosine similarity against existing entries; a similarity of 0.95 or above flags the upload as a likely duplicate rather than silently indexing a redundant copy.

## Phase 3 — Intent-Aware Query Understanding

Natural-language queries are parsed on-device by Phi-3 Mini via Ollama into structured topic/keyword intent — no cloud dependency, no closed-set category limit.

## Phase 4 — Hybrid Retrieval & Reranking

Dense semantic search and sparse/keyword (BM25-style) search run in parallel inside Qdrant's native hybrid query; the two rankings are merged using Reciprocal Rank Fusion ( $\text{score}(d) = \sum 1/(k + \text{rank}_i(d))$ ,  $k=60$ ) rather than a single opaque hybrid score, and the fused candidate set is reordered by a local cross-encoder before the top-3 results are shown. Each result is returned with its single highest-similarity sentence highlighted, giving a human-interpretable “why this matched,” and if the top fused-and-reranked score falls below a tuned confidence threshold, the system responds with “no confident match found” instead of forcing a low-relevance top-3.

## Phase 5 — Secure Storage & Remote Delivery

Files, metadata, and the vector index persist on the Pi 4B + USB storage; Cloudflare Tunnel exposes one HTTPS endpoint with no port-forwarding or static IP.

## Phase 6 — Evaluation

- Test corpus: 150–200 files across 5–8 topic categories.
- Query set: 30–50 natural-language queries with an independently authored ground-truth target file each.
- Metrics: top-3 retrieval accuracy, average end-to-end latency including reranking, and accuracy delta vs. keyword-only and dense-only baselines.
- Results reported as a comparison table/chart at Review-3, directly answering “how do we know hybrid retrieval actually helped?”

## 7.7 Week-by-Week Build Plan (Lab Manual)

This is the operational build plan for Phase 1, mapped onto the department's weekly reporting calendar (§11.1). Each week lists an objective, exact setup commands, concrete tasks, a deliverable, and an approximate cumulative completion percentage against the template's indicative scale (§11.2). Today's date falls inside Week 3 — that week is the active one; Weeks 1–2 are complete; Weeks 4–9 are upcoming. No hardware is required until Week 6.

### Week 1 (03–08 Aug 2026) • Done

*Deliver Review-0 while simultaneously bootstrapping the full development environment on all three laptops — implementation starts this week, in parallel with the paperwork, not after it.*

#### Setup commands:

```
npx create-next-app@latest intentcloud-web --typescript --tailwind --eslint --app
cd intentcloud-web && npm run dev
python -m venv venv && source venv/bin/activate
pip install fastapi "uvicorn[standard]" pymupdf python-docx sentence-transformers qdrant-client
curl -fsSL https://ollama.com/install.sh | sh # or Ollama.app on macOS/Windows
ollama pull phi3:mini
```

#### Tasks:

- Deliver Review-0: problem statement, existing-system analysis, and literature-survey findings presented to and approved by the panel.
- Scaffold the Next.js app (App Router, TypeScript, Tailwind) and confirm it renders at localhost:3000.
- Scaffold a FastAPI app with a GET /health endpoint returning {"status": "ok"}; enable CORS for localhost:3000.
- Confirm the Next.js home page can fetch() /health and display the response — proves the two codebases can talk to each other.
- Test Ollama + phi3:mini locally with a sample prompt from the command line (ollama run phi3:mini "...").
- Install qdrant-client, create a local embedded collection, insert 3 dummy vectors, run one test query.

#### Individual Tasks (Named):

- **Sujith Putta** — scaffold the FastAPI GET /health endpoint with CORS enabled for localhost:3000; test Ollama + phi3:mini from the command line; confirm the Ollama backend correctly auto-detects Metal/MPS on his Mac.
- **K Vikas Aneesh Reddy** — install qdrant-client, create a local embedded Qdrant collection, insert 3 dummy vectors, and run one test query; draft a shortlist of Raspberry Pi 4B vendors/prices for the Week-5 hardware order.
- **Mokshith Karnati** — scaffold the Next.js app (App Router, TypeScript, Tailwind) and confirm fetch() from the home page to /health renders correctly; prepare and deliver the Review-0 slide deck.

**Deliverable:** Review-0 delivered and approved; Next.js and FastAPI verified talking to each other locally; Ollama and Qdrant both verified working on every team member's laptop.

**Cumulative target:** ~8% — **Proposal Complete band, with implementation already underway**

## Week 2 (10–14 Aug 2026) • Done

*Finalize the literature survey while building Phase 1 (Data Ingestion & Preprocessing) for real — the upload pipeline goes live this week.*

### Tasks:

- Finalize the 19-paper IEEE-format literature survey and lock the research-gap table and motivation section (§6).
- Build the Next.js /upload page: a drag-and-drop file input (PDF/DOCX/TXT) that POSTs to the FastAPI backend.
- Build FastAPI POST /upload: accept a multipart file, save it under a ./storage layout that mirrors the eventual Pi path structure.
- Implement text extraction — PyMuPDF for PDF, python-docx for DOCX, plain read for TXT.
- Run the extraction script over 5–10 sample files and confirm sane extracted-text lengths.

### Individual Tasks (Named):

- **Sujith Putta** — implement PyMuPDF (PDF) and python-docx (DOCX) text extraction functions and unit-test them across 5–10 sample files, checking for sane extracted-text lengths.
- **K Vikas Aneesh Reddy** — build the FastAPI POST /upload endpoint accepting a multipart file and saving it under a ./storage layout that mirrors the eventual Pi path structure.
- **Mokshith Karnati** — build the Next.js /upload page (drag-and-drop PDF/DOCX/TXT input posting to FastAPI); finalize the 19-paper literature survey formatting and the research-gap table (§6).

**Deliverable:** Literature survey finalized; a user can upload a file from the Next.js UI and receive back the extracted text length/preview as JSON.

**Cumulative target:** ~18% — **Literature Survey complete, Phase 1 implementation underway**

## Week 3 (17–22 Aug 2026 — CURRENT WEEK) • In Progress

*Complete Phase 2 (Semantic Representation & Indexing) on the laptop; finish the Review-1 material. Laptop-only — no hardware needed yet.*

### Setup commands:

```
pip install sentence-transformers # confirm version, already installed in Week 1
```

### Tasks:

- Generate embeddings for uploaded document text using sentence-transformers (all-MiniLM-L6-v2).
- Create a local Qdrant collection matching the Pi-target schema: id, vector, payload {filename, topic\_tags, upload\_time}.
- Wire the upload endpoint end-to-end: extract → embed → upsert into Qdrant.
- Build the Next.js /dashboard page: file count and topic tag cloud, sourced from a new GET /stats endpoint.
- Finalize the Review-1 architecture and workflow diagrams (Qdrant-hybrid version, §5) and draft the slide outline.
- List the target 150–200-file test corpus (topics/sources) so Phase-6 evaluation isn't a last-minute scramble.

### Individual Tasks (Named):

- **Sujith Putta** — implement sentence-transformers (all-MiniLM-L6-v2) embedding generation and wire it into the /upload endpoint (extract → embed); write the Review-1 module-design section covering the cognitive-layer components.
- **K Vikas Aneesh Reddy** — create the local Qdrant collection matching the Pi-target schema (id, vector, payload {filename, topic\_tags, upload\_time}); upsert embeddings on every upload and confirm a similarity query returns the correct file.
- **Mokshith Karnati** — build the Next.js /dashboard page (file count + topic tag cloud) against a new GET /stats endpoint; finalize the Review-1 architecture and workflow diagrams (Qdrant-hybrid version, §5) and draft the slide outline.

**Deliverable:** Uploading a file end-to-end produces a searchable Qdrant entry; the dashboard shows a live file count; Review-1 deck drafted.

**Cumulative target:** ~30% — **entering Stage-1 (Detailed Design & Architecture)**

## Week 4 (24–29 Aug 2026) • Upcoming

*Deliver Review-1; build Phase 3 (Intent-Aware Query Understanding).*

### Tasks:

- Deliver the Review-1 presentation (29 Aug): architecture, module design, workflow diagrams, DB/index design, tech stack, task allocation.
- Implement POST /search: send the free-text query to Ollama/phi3:mini and parse the structured intent JSON (topic/keywords).
- Build the Next.js /search page: a search bar that calls /search and renders raw top-N results (no reranking yet).
- Regression-test the Phase 1–2 pipeline (upload → extract → embed → store) against the growing test corpus.

#### Individual Tasks (Named):

- **Sujith Putta** — implement POST /search, sending the free-text query to Ollama/phi3:mini and parsing the structured intent JSON (topic/keywords); add the Tesseract OCR fallback into the extraction pipeline for scanned PDFs (\$5.4).
- **K Vikas Aneesh Reddy** — regression-test the Phase 1–2 pipeline (upload → extract → embed → store) against the growing test corpus; implement the cosine-similarity duplicate-detection check at upload time (\$5.4).
- **Mokshith Karnati** — build the Next.js /search page — a search bar that calls /search and renders raw top-N dense-only results; prepare and deliver the Review-1 presentation (29 Aug).

**Deliverable:** Review-1 delivered; natural-language search returns raw dense-only results in the browser.

**Cumulative target:** ~40% — **Stage-1 continues**

#### Week 5 (31 Aug–04 Sep 2026) • Upcoming

*Build Phase 4 (Hybrid Retrieval & Reranking) on the laptop; place the Raspberry Pi hardware order.*

#### Tasks:

- Implement sparse/BM25-style keyword scoring to run alongside dense search.
- Wire Qdrant's native hybrid query so dense and sparse scores are fused in a single call.
- Add a cross-encoder reranking step over the merged top-N candidates, returning the final top-3.
- Update the Next.js /search page to show the reranked top-3 with a relevance explanation.
- Place the Raspberry Pi hardware order (\$10.3) — target by 5 September so it arrives before Week 6.

#### Individual Tasks (Named):

- **Sujith Putta** — implement sparse/BM25-style keyword scoring alongside dense search, and implement Reciprocal Rank Fusion (RRF) to merge the dense and sparse rankings ahead of reranking (\$5.4).
- **K Vikas Aneesh Reddy** — wire Qdrant's native hybrid query call and add the cross-encoder (ms-marco-MiniLM) reranking step over the fused top-N candidates; place the Raspberry Pi hardware order (\$10.3) by 5 September.
- **Mokshith Karnati** — update the Next.js /search page to show the reranked top-3 with the highlighted matched-snippet explanation (\$5.4); draft the 30–50 query evaluation set with independently authored ground-truth targets.

**Deliverable:** Hybrid, reranked top-3 search working end-to-end on the laptop simulation; Pi hardware ordered.

**Cumulative target:** ~48% — **Stage-2 (Methodology & Partial Implementation) begins**

#### Week 6 (07–12 Sep 2026) • Upcoming

*Deliver Review-2 (≥25% implementation required) and submit the research paper; unbox the Pi and migrate the memory layer onto it.*

#### Setup commands:

```
sudo apt update && sudo apt install -y python3-pip python3-venv
sudo raspi-config # enable SSH, set hostname (e.g. intentcloud-pi)
```

#### Tasks:

- Prepare and deliver the Review-2 demo: progress status plus the hybrid+reranked pipeline running live — well above the 25% bar.
- Submit the research paper / conference-journal contribution (due 12 Sep).
- Flash Raspberry Pi OS (64-bit, Bookworm) to the microSD card; complete headless setup (SSH, static hostname).
- Install Python, Qdrant, and FastAPI on the Pi; migrate the Qdrant index and file storage from the laptop to the Pi.
- Confirm the Pi is reachable from the laptop over the local network end-to-end.

#### Individual Tasks (Named):

- **Sujith Putta** — prepare the Review-2 demo script; tune the confidence threshold for the “no confident match” fallback (\$5.4); write the methodology section of the research paper.
- **K Vikas Aneesh Reddy** — flash Raspberry Pi OS, complete headless SSH setup, install Python/Qdrant/FastAPI on the Pi, and migrate the Qdrant index and file storage from the laptop to the Pi.
- **Mokshith Karnati** — finalize and submit the research paper (due 12 Sep); confirm the statically-exported Next.js UI serves correctly from the Pi and that the Pi is reachable from the laptop over the LAN.

**Deliverable:** Review-2 delivered; research paper submitted; hybrid search running live from the Pi over the LAN.

**Cumulative target:** ~58% — **satisfies the Review-2 ≥25% requirement with margin**



**Week 7** (14–18 Sep 2026) • Upcoming

*Stand up remote access and finish Phase 5 (Secure Storage & Remote Delivery).*

**Setup commands:**

```
cloudflared tunnel login
cloudflared tunnel create intentcloud
cloudflared tunnel route dns intentcloud <subdomain>.example.com
```

**Tasks:**

- Install cloudflared on the Pi, create the named tunnel, and confirm the Next.js UI and API are reachable via the public HTTPS URL.
- Implement the download flow: clicking a result card in Next.js calls GET /download/{id}, serving the file from Pi/USB storage.
- Run a full upload → search → download smoke test entirely through the public tunnel URL, from a phone on a different network.
- Begin drafting the Review-3 slide deck and demo script.

**Individual Tasks (Named):**

- **Sujith Putta** — validate that the hybrid + RRF + rerank pipeline behaves identically once Qdrant is running on the Pi rather than the laptop; document the cross-platform inference behavior (MPS/CUDA/CPU) for the Phase-1 report.
- **K Vikas Aneesh Reddy** — install cloudflared on the Pi, create the named tunnel, and confirm public HTTPS reachability; implement GET /download/{id} serving the file from Pi/USB storage.
- **Mokshith Karnati** — run the full upload → search → download smoke test through the public tunnel URL from a phone on a different network; begin drafting the Review-3 slide deck and demo script.

**Deliverable:** Full upload → search → download loop working end-to-end from the Pi via the public tunnel URL.

**Cumulative target:** ~68% — **Stage-2 complete, entering Stage-3**

**Week 8** (21–26 Sep 2026) • Upcoming

*Deliver Review-3 (≥50% implementation required); run Phase 6 (Evaluation).*

**Tasks:**

- Prepare and deliver the Review-3 demo (22 Sep): 50%+ implementation, methodology, results & discussion, conclusion.
- Build the evaluation harness: run the 30–50 query set against keyword-only, dense-only, and hybrid+rerank pipelines; log top-3 hit/miss and latency.
- Run the first full evaluation pass and capture baseline numbers for the Review-3 report.
- Fix bugs surfaced by the evaluation run (edge cases in extraction, reranking ties, timeout handling).

**Individual Tasks (Named):**

- **Sujith Putta** — build the evaluation harness scripts that run the 30–50 query set against keyword-only, dense-only, and hybrid+RRF+rerank pipelines, logging top-3 hit/miss and latency.
- **K Vikas Aneesh Reddy** — fix backend bugs surfaced by the evaluation run — extraction edge cases, reranking ties, timeout handling — and confirm duplicate-detection isn't rejecting legitimately similar-but-distinct files.
- **Mokshith Karnati** — prepare and deliver the Review-3 demo (22 Sep); build the accuracy/latency comparison chart into the Next.js /dashboard for the results discussion.

**Deliverable:** Review-3 delivered; first benchmark numbers (accuracy, latency) in hand.

**Cumulative target:** ~80% — **satisfies the Review-3 ≥50% requirement with margin**

**Week 9** (28 Sep–02 Oct 2026) • Upcoming

*Finalize evaluation and submit the consolidated Phase-1 report.*

**Tasks:**

- Finalize evaluation results — top-3 accuracy %, latency, and accuracy deltas vs. both baselines — and build the comparison chart.
- Write and assemble the consolidated Phase-1 report: methodology, results & discussion, conclusion, references.
- Final QA pass: fresh-install test on the Pi from a clean SD-card image to confirm the system is reproducible.
- Submit the Phase-1 report by 28 September 2026.

**Individual Tasks (Named):**

- **Sujith Putta** — finalize top-3 accuracy, latency, and accuracy-delta numbers vs. both baselines; write the Results & Discussion section of the Phase-1 report.
- **K Vikas Aneesh Reddy** — run the final QA pass — a fresh-install test on the Pi from a clean SD-card image — to confirm the system is reproducible end-to-end.
- **Mokshith Karnati** — assemble and format the consolidated Phase-1 report (methodology, results & discussion, conclusion, references) and submit it by 28 September 2026.

**Deliverable:** Phase-1 report submitted; system demonstrably reproducible from a clean Pi image.  
**Cumulative target:** ~90% — Phase-1 scope fully delivered with margin; remainder carries into Phase 2

8. Deferred Features (Not in Phase-1 Demo)

| Feature                                      | Reason Deferred                                           |
|----------------------------------------------|-----------------------------------------------------------|
| Neo4j knowledge graph for file relationships | Adds complexity without changing the core demo            |
| Multi-node Raspberry Pi cluster              | A single Pi is sufficient to demonstrate the concept      |
| Voice input                                  | Text input is sufficient; a microphone adds hardware risk |
| User authentication / login                  | Out of scope for the capstone demo                        |
| File versioning                              | Non-essential within the 1.5-month implementation window  |
| Mobile-optimized UI                          | Desktop-browser access is sufficient for evaluation       |

9. Software Stack & Versions

| Component              | Tool / Library                      | Version                        | Purpose                                                                                           |
|------------------------|-------------------------------------|--------------------------------|---------------------------------------------------------------------------------------------------|
| Operating System       | Raspberry Pi OS (64-bit)            | Bookworm (2026-01)             | Host OS for the edge/memory-layer node                                                            |
| Runtime                | Python                              | 3.11.x                         | Core application language, laptop + Pi                                                            |
| LLM Runtime            | Ollama                              | 0.5.x                          | Serves the local LLM without cloud calls                                                          |
| Hardware Acceleration  | Ollama backend / PyTorch MPS / CUDA | auto-detect                    | Selects Apple Silicon MPS (Mac), NVIDIA CUDA (Windows/Linux GPU), or CPU automatically at runtime |
| Intent-Parsing Model   | Phi-3-mini-4k-instruct              | Q4_K_M (GGUF)                  | Parses free-text queries into structured intent JSON                                              |
| Embeddings             | sentence-transformers               | 3.x (all-MiniLM-L6-v2)         | Converts document/query text to 384-d dense vectors                                               |
| Reranker               | Cross-encoder (ms-marco-MiniLM)     | latest (sentence-transformers) | Reorders hybrid candidates for top-3 accuracy                                                     |
| Vector + Hybrid Index  | Qdrant (embedded mode)              | 1.9.x                          | Stores dense+sparse vectors, metadata, on-Pi hybrid search                                        |
| Text Extraction        | PyMuPDF                             | 1.24.x                         | Extracts text from PDF uploads                                                                    |
| Text Extraction        | python-docx                         | 1.1.x                          | Extracts text from DOCX uploads                                                                   |
| OCR Fallback           | Tesseract (pytesseract)             | 5.x                            | Extracts text from scanned/image-only PDF pages that PyMuPDF cannot read directly (\$5.4)         |
| Backend – Laptop       | FastAPI + Uvicorn                   | 0.11x / 0.30x                  | AI orchestration service (intent, embed, rerank)                                                  |
| Backend – Pi           | FastAPI + Uvicorn                   | 0.11x / 0.30x                  | File upload/serving, routing, UI hosting                                                          |
| Frontend Framework     | Next.js (App Router)                | 15.x / React 19                | Upload form, search bar, result cards, memory dashboard                                           |
| Styling                | Tailwind CSS                        | 3.4.x                          | Utility-first styling for the Next.js UI                                                          |
| Frontend Build Runtime | Node.js                             | 20.x LTS                       | Build tooling for Next.js (dev laptop only, not required on Pi)                                   |
| Remote Access          | Cloudflare Tunnel (cloudflared)     | 2026.x                         | Free public HTTPS URL, no port-forwarding                                                         |
| Version Control        | Git / GitHub                        | latest                         | Source control and team collaboration                                                             |

10. Hardware Requirements & Budget

The hardware satisfies the capstone's 50% hardware-implementation requirement: the Raspberry Pi is the physical storage node, the always-on access point, and the global network node — the system has no persistent storage or public reachability without it.

10.1 Hardware

| Item                   | Spec                                    | Qty | Unit Cost (₹) | Total (₹) |
|------------------------|-----------------------------------------|-----|---------------|-----------|
| Raspberry Pi 4 Model B | 8 GB RAM (memory-layer / edge node)     | 1   | 6,500         | 6,500     |
| MicroSD Card           | 64 GB, A2/U3 (boot + Pi OS)             | 1   | 700           | 700       |
| USB Storage Drive      | 128 GB USB 3.0 pen drive (file storage) | 1   | 1,000         | 1,000     |
| USB-C Power Supply     | 5.1V / 3A official Pi adapter           | 1   | 700           | 700       |



| Item                     | Spec                                             | Qty | Unit Cost (₹) | Total (₹) |
|--------------------------|--------------------------------------------------|-----|---------------|-----------|
| Cooling Case + Heatsinks | Active-cooling case for sustained inference load | 1   | 600           | 600       |
| Ethernet Cable (Cat6)    | For stable laptop ↔ Pi local network link        | 1   | 200           | 200       |
| Miscellaneous            | Cables, card reader, contingency                 | —   | 500           | 500       |
| Estimated Total          |                                                  |     |               | 10,200    |

## 10.2 Software / Service Cost

Every software component is open-source and self-hosted, or free-tier; the only recurring cost is electricity to keep the Pi powered.

| Item                                                                             | Spec                                         | Qty | Unit Cost (₹) | Total (₹) |
|----------------------------------------------------------------------------------|----------------------------------------------|-----|---------------|-----------|
| Ollama, Phi-3 Mini, Qdrant, FastAPI, sentence-transformers, PyMuPDF, python-docx | Open-source, self-hosted                     | —   | 0             | 0         |
| Cloudflare Tunnel                                                                | Free tier, sufficient for demo-scale traffic | —   | 0             | 0         |
| GitHub (private repo)                                                            | Free for academic/team use                   | —   | 0             | 0         |
| Estimated Total                                                                  |                                              |     |               | 0         |

**Total project budget: ₹10,200 (one-time, hardware only). No cloud, API, or licensing costs.**

## 10.3 Hardware Procurement Plan

As of this document's date, no hardware has been purchased. Procurement is deliberately deferred to September so that Weeks 1–5 (through end-August, see §7.7) are spent entirely on software that runs identically on the laptop and, later, on the Pi — no team member is blocked by the Pi's absence.

- Target purchase window: 01–05 September 2026 (start of Week 5), so the Pi is bought, flashed, and bench-tested before the Review-2 demo on 12 September 2026.
- Owner: Sujith Putta places the order (Amazon / local electronics vendor in Bangalore, e.g. Robu.in or SP Road) and shares the invoice with the guide for reimbursement records.
- Interim strategy while hardware is in transit: Qdrant, FastAPI, and the USB-drive file store all run locally on the development laptop inside a folder that mirrors the Pi's exact directory layout, so the migration in Week 6 is a copy-and-restart, not a rewrite.
- Fallback buffer: ordering by 5 September leaves a full week of slack before Review-2 in case of delivery delay; if the Pi has not arrived by 10 September, the Review-2 demo runs on the laptop-only simulation and the note is disclosed to the panel.

## 11. Project Schedule (per Capstone Phase-1 Circular)

Dates below follow the official department circular. “Done” milestones have already been submitted; the team's current focus is Review-1 (System Architecture & Design), due 29 August 2026.

| Milestone                  | Deliverable                                                                                                    | Date        | Status      |
|----------------------------|----------------------------------------------------------------------------------------------------------------|-------------|-------------|
| Team Formation             | 3-member team finalized                                                                                        | 26 May 2026 | Done        |
| Guide Allocation           | Dr. Ramandeep Kaur assigned as guide                                                                           | 28 May 2026 | Done        |
| Domain & Problem Statement | Problem statement approved (20 marks)                                                                          | 10 Jun 2026 | Done        |
| Literature Survey          | 19-paper IEEE-format survey, research gap identified (20 marks)                                                | 7 Jul 2026  | Done        |
| Review-0                   | Problem statement, objectives, existing-system analysis, literature survey presentation (50 marks)             | 8 Aug 2026  | Done        |
| Review-1                   | System architecture, module design, workflow diagrams, DB/index design, tech stack, task allocation (50 marks) | 29 Aug 2026 | In Progress |
| Review-2                   | Minimum 25% implementation demo + progress status (50 marks)                                                   | 12 Sep 2026 | Pending     |
| Research Contribution      | Paper submission to IEEE conference/journal                                                                    | 12 Sep 2026 | Pending     |
| Review-3                   | 50% implementation demo, methodology, results & discussion, conclusion (50 marks)                              | 22 Sep 2026 | Pending     |
| Phase-1 Report Submission  | Consolidated Phase-1 report (50 marks)                                                                         | 28 Sep 2026 | Pending     |

### 11.1 Weekly Progress Reporting — Full Circular Calendar

Weekly reports are mandatory every Saturday, per the official department circular reproduced below. The first three weeks were due together on 22 August 2026; every week after that is submitted individually, on or before the end date of its window. Weeks 1–9 fall inside

Phase-1 core delivery (§7.7); weeks 10–17 continue the same reporting cadence into the post-Phase-1 buffer / Phase-2 lead-in and will be scoped once Phase-2 guidance is issued.

| Week | Reporting Window | Week | Reporting Window |
|------|------------------|------|------------------|
| 1    | 03–08 Aug        | 2    | 10–14 Aug        |
| 3    | 17–22 Aug        | 4    | 24–29 Aug        |
| 5    | 31 Aug–04 Sep    | 6    | 07–12 Sep        |
| 7    | 14–18 Sep        | 8    | 21–26 Sep        |
| 9    | 28 Sep–02 Oct    | 10   | 05–10 Oct        |
| 11   | 12–16 Oct        | 12   | 19–24 Oct        |
| 13   | 26–31 Oct        | 14   | 02–06 Nov        |
| 15   | 09–14 Nov        | 16   | 16–20 Nov        |
| 17   | 23–26 Nov        |      |                  |

## 11.2 Weekly Report Template — Required Fields

Every submission must follow the official “B.Tech Final Year Capstone Project Phase 1 — Weekly Progress Report” template exactly. Each week's document must contain:

- Header block: Project Semester (Aug–Dec 2026), Week No., Date range (From/To), Team Number, Names & USNs of all team members, Name of the Guide, Project Title.
- Work Carried Out by the Team During the Week — bullet points only, filled in by the whole team together.
- Individual Contribution from the Student — filled in separately by each of the three members (Sujith, Vikas, Mokshith), describing their own specific work.
- Work Planned for the Following Week — bullet points.
- Major Bottlenecks (if any).
- Percentage Work Completed — a single number, positioned against the indicative scale below.
- Signature of the Students and Signature of the Guide(s) — both required before submission.

Indicative activity-to-percentage mapping (as issued by the department; used to justify the % reported each week):

| Particular                                                                                           | Percentage |
|------------------------------------------------------------------------------------------------------|------------|
| Proposal Complete — Domain & Problem identification                                                  | 0–5%       |
| Project Design Phase-1 & Literature Survey + Initial Implementation + Initial Thesis                 | 6–30%      |
| Stage-1: Project Detailed Design and Architecture                                                    | 31–60%     |
| Stage-2: Methodology & Partial Implementation                                                        | 61–75%     |
| Stage-3: 50% Implementation + Detailed Test Cases                                                    | 76–90%     |
| Project Final Evaluation: Final Results, Discussion, Research Contribution & Final Report Submission | 91–100%    |

## 12. Team, Responsibilities & Immediate Action Items

### 12.1 Start Now — Week 3 (17–22 Aug 2026)

The team is currently inside Week 3 of the reporting calendar. Scaffolding and core-tool verification (Next.js, FastAPI, Ollama, Qdrant) were completed in Weeks 1–2 (§7.7); this week's focus is Phase 2 (Semantic Representation & Indexing) and closing out the Review-1 material. No hardware has been purchased yet — every task below runs on a laptop only, so nobody is blocked waiting on the Pi. These are due by the end of this week's window (22 Aug), ahead of Review-1 on 29 Aug. (The same three named tasks are repeated here from §7.7 Week 3 for quick reference; §7.7 now carries this same individual-task breakdown, by name, for every week 1–9, not only the current one.)

- Sujith Putta — implement sentence-transformers (all-MiniLM-L6-v2) embedding generation for uploaded document text and wire it into the /upload endpoint (extract → embed); write the Review-1 module-design section covering the cognitive-layer components.
- K Vikas Aneesh Reddy — create the local Qdrant collection matching the Pi-target schema (id, vector, payload {filename, topic\_tags, upload\_time}); upsert embeddings on every upload and confirm a similarity query returns the correct file.
- Mokshith Karnati — build the Next.js /dashboard page (file count + topic tag cloud) against a new GET /stats endpoint; finalize the Review-1 architecture and workflow diagrams (Qdrant-hybrid version, §5) and draft the Review-1 slide outline.
- All three — fill and submit the Week-3 progress report (§11.2 template fields) by Saturday, and place the Raspberry Pi hardware order (§10.3) no later than 5 September.

## 12.2 Roles & Ongoing Responsibilities

| Name                      | USN         | Primary Focus                                                                            |
|---------------------------|-------------|------------------------------------------------------------------------------------------|
| Sujith Putta              | ENG23CT0058 | Cognitive layer: intent parsing, embeddings, hybrid retrieval & reranking; documentation |
| K Vikas Aneesh Reddy      | ENG23CT0052 | Memory layer: Qdrant indexing, FastAPI backend, Pi/USB storage setup                     |
| Mokshith Karnati          | ENG23CT0053 | Next.js + Tailwind UI, Cloudflare Tunnel deployment, evaluation & benchmarking           |
| Guide: Dr. Ramandeep Kaur | —           | Review, approval, and supervision at every stage per department policy                   |

## 13. Key Risks & Mitigations

| Risk                             | Impact                                                  | Mitigation                                                   |
|----------------------------------|---------------------------------------------------------|--------------------------------------------------------------|
| CPU-only inference latency       | Query response exceeds the ~15s target                  | Q4 quantized Phi-3 Mini; cap reranking candidates            |
| Pi memory limits at scale        | Index growth beyond 200 files could degrade performance | Corpus capped at 150–200 files for Phase-1 evaluation        |
| Single point of hardware failure | Pi/USB failure halts the demo                           | Nightly backup of index + files to laptop during development |
| Tunnel/network dependency        | No internet at demo venue breaks remote access          | Local-network fallback demo (same Wi-Fi, no tunnel needed)   |

## 14. Response to Review-0 Feedback (38/50)

Review-0 scored 38/50. Rather than treat that as a final verdict, the team used it to drive the changes in this Version 4.0 document. The gaps inferred from the score and panel discussion, and the concrete response to each, are:

- **Shallow methodology beyond “embed and search.”** Addressed in §5.4 with five concrete additions — OCR fallback, duplicate detection, Reciprocal Rank Fusion for hybrid score merging, explainable snippet highlighting, and a confidence-threshold no-match response — each traced to a specific literature gap in §6.
- **No justification for why the system works on different team members’ laptops.** Addressed with an explicit hardware-acceleration backend (§5.1, §9): Apple Silicon MPS/Metal on Mac, NVIDIA CUDA on Windows, CPU fallback otherwise, verified as a Week-1 setup task (§7.7) rather than assumed.
- **Weekly plan read as a team-level checklist with no individual accountability.** Every week in §7.7 (Weeks 1–9) now names all three members and states precisely what each is building that week, in addition to the shared team tasks and setup commands.
- **Hybrid retrieval described too vaguely (“Qdrant fuses the scores”).** Replaced with the explicit Reciprocal Rank Fusion formula (§5.4, Phase 4 in §7) so the fusion step is a stated, citable algorithm rather than a black box.
- **No mechanism to prevent low-relevance results from being shown as if they were correct.** Addressed with the confidence-threshold “no confident match” fallback (§5.4), so the system fails visibly instead of silently returning an irrelevant top-3.

These changes do not alter the core objectives, architecture, or Phase-1 schedule approved after Review-0; they deepen the technical substance underneath decisions that were previously stated at a summary level.

## 15. References

Complete 19-reference IEEE-format bibliography, drawn from the literature survey (§6), numbered [1]–[19] as cited throughout this document:

- [1] K. Karthikeyan, S. Kishorekumar, C. M. V. Prasad, and V. P. Kumar, “Smart Document Search Engine Leveraging Automated Metadata Extraction For Intelligent File Discovery,” Proc. 2025 IEEE ICINVENTS, 2025, pp. 1–6.
- [2] A. Vaswani et al., “Attention Is All You Need,” Proc. 31st NeurIPS, 2017, pp. 5998–6008.
- [3] V. N. R. K, K. Vasana, J. Jomy, S. A, S. Rose, and A. Suresh, “Semantic Information Retrieval using Transformer-based Embeddings and Vector Similarity Search,” Proc. 7th ICIDCA, 2025, pp. 1031–1037.
- [4] Brundha J. and K. N. Meera, “Vector Model Based Information Retrieval System With Word Embedding Transformation,” Proc. ICETET-SIP-22, 2022, pp. 1–4.
- [5] M. Douze et al., “The Faiss library,” arXiv:2401.08281, 2024.
- [6] S. Krishnan and N. Subramanian, “Semantic Search in a Document Management System Using Retrieval Augmented Generation,” Proc. IEEE/ACIS SNPD, 2025, pp. 6–10.

- [7] T. P. Jacob, B. L. S. Bizotto, and M. Sathiyarayanan, "Constructing the ChatGPT for PDF Files with LangChain – AI," Proc. IEEE ICICT, 2024, pp. 835–839.
- [8] K. Sawarkar, A. Mangal, and S. R. Solanki, "Blended RAG: Improving RAG Accuracy with Semantic Search and Hybrid Query-Based Retrievers," arXiv:2404.07220v2, 2024.
- [9] S. Rafi et al., "AI Document Search Chatbot Using Retrieval-Augmented Generation (RAG)," Proc. IEEE I5CPS, 2026, pp. 1–4.
- [10] M. Tiwari, J. Kaur, A. Harsh, H. Kaur, and R. S. Surin, "Fine-Tuning T5 and BERT for Effective Question Answering and Document Retrieval: A Transfer Learning Approach," Proc. IEEE WCCST, 2026, pp. 82–88.
- [11] B. Saha, U. Saha, and M. Z. Malik, "QuIM-RAG: Advancing Retrieval-Augmented Generation With Inverted Question Matching for Enhanced QA Performance," IEEE Access, vol. 12, pp. 185401–185410, 2024.
- [12] O. A. Cejas, Y. Guo, and Q. Tang, "From Retrieval to Response: Tracing the Impact of Embedding Quality in RAG Systems," IEEE Access, vol. 13, pp. 212773–212781, 2025.
- [13] S. Sriram, C. H. Karthikeya, K. P. K. Kumar, N. Vijayaraj, and T. Murugan, "Leveraging Local LLMs for Secure In-System Task Automation With Prompt-Based Agent Classification," IEEE Access, vol. 12, pp. 177038–177049, 2024.
- [14] S. Kuchlous and M. Kadaba, "Short Text Intent Classification for Conversational Agents," Proc. IEEE 17th INDICON, 2020, pp. 1–4.
- [15] F. D. Puspitasari et al., "Efficient RAG with Intent-Aware Retrieval and Semantics-Preserving Chunking," arXiv:2606.01240, 2026.
- [16] Y. Nan et al., "Research on an Intelligent File Query System Model Integrating Natural Language Understanding and Information Retrieval," Proc. IEEE ICPEGE, 2026, pp. 676–680.
- [17] V. Holubiev et al., "Revolutionizing Contact Center Knowledge Management: the Game-Changing Role of AI Large Language Models and Autonomous Agents in Text Organization and Optimization," Proc. IEEE ACIT, 2024, pp. 778–783.
- [18] J. Nickolls, I. Buck, M. Garland, and K. Skadron, "Scalable parallel programming with CUDA," Queue, vol. 6, no. 2, pp. 40–53, 2008.
- [19] Apple Inc., "Metal Performance Shaders: Accelerating compute and graphics on Apple hardware," Apple Developer Documentation, 2026.